Распределённая обработка графов: масштабируемость D-Galois

Дмитриевцев Алексей Сергеевич, группа 23.Б-11

03.11.2026

Окружение и методика

Стенд:

- 8 облачных машин в одной подсети: по 4 vCPU (2 ядра × 2 потока), 15 ГБ; 5 машин — AMD EPYC 9754 (в том числе машина для замеров на 1 хосте), 3 — EPYC 7763.
- Задержка сети между машинами (ping): в среднем меньше 1 мс.
- Ubuntu 24.04, Open MPI 4.1.6, Galois (коммит `b67f942`), GCC 13.
- Хост — машина: один MPI-процесс, 3 рабочих потока и поток обмена Galois.

Методика

- 1, 2, 4, 8 машин; режимы BSP (`-exec=Sync`) и BASP (`-exec=Async`); разбиение OEC.
- 3 прогона, столбец — медиана.
- Метрики — из статистики Galois: время, синхронизация, ожидание, раунды, байты, число копий вершин.

Графы и устройство приложений

Граф	Тип	$\|V\|$	$\|E\|$	ср. степень	макс. степень
kron_g500-logn21	соцсеть	2^{21}	31.8M	30.3	102.5K
urand_n22	случайный	2^{22}	33.6M	16.0	40
rgg_n_2_22_s0	геометрический	2^{22}	30.4M	14.5	38
roadNet-CA	дороги	2.0M	2.8M	2.8	12
road_n22	дороги	2^{22}	5.4M	2.6	5

Синтетика: число вершин — параметр генератора (2^{21}, 2^{22}), генераторы как в блоке 1, номера вершин перемешаны случайно; roadNet-CA — реальная дорожная сеть. SSSP — веса 1..100; источник — вершина максимальной степени.

- **BFS, SSSP (push)**: раунд проходит все вершины хоста и обновляет соседей активных; копии вершины на других хостах сводятся к основной по минимуму.
- **PageRank (pull)**: каждая вершина забирает изменения рангов входящих соседей; итерация проходит все рёбра, копии сводятся суммой.
- **TC**: хост получает полные списки соседей своих вершин; обменов во время счёта нет.

Как устроен раунд BFS в D-Galois

Разбиение (OEC, при загрузке графа):

- Машина берёт непрерывный диапазон номеров вершин, рёбра поделены поровну.
- Ребро в чужой диапазон — копия вершины; копии синхронизирует Gluon.

Раунд (BSP):

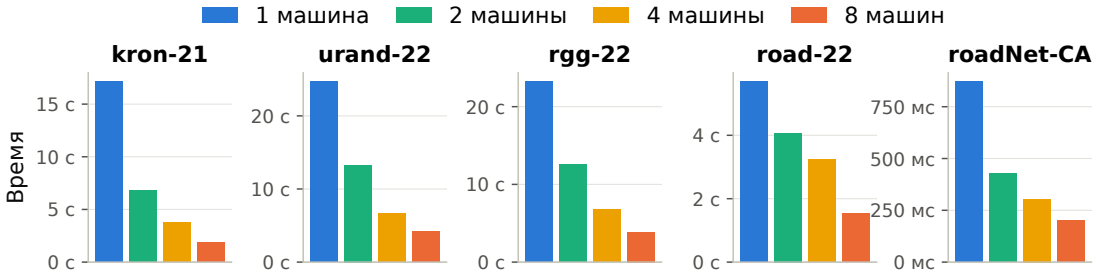
1. Проход по *всем* вершинам машины; активные обновляют соседей (`atomicMin`).
2. Gluon сводит копии по минимуму, машина ждёт сообщений остальных.
3. `MPI_Allreduce` числа активных вершин: 0 — конец.

Граф	Уровней	Раундов на	
		1 маш.	8 маш.
kron-21	4	5	6
urand-22	7	8	8
rgg-22	1328	498	1099
road-22	3418	1640	3159
roadNet-CA	726	393	407

- Внутри машины фронт за раунд проходит несколько уровней.
- Через границу машин — только в следующем раунде.

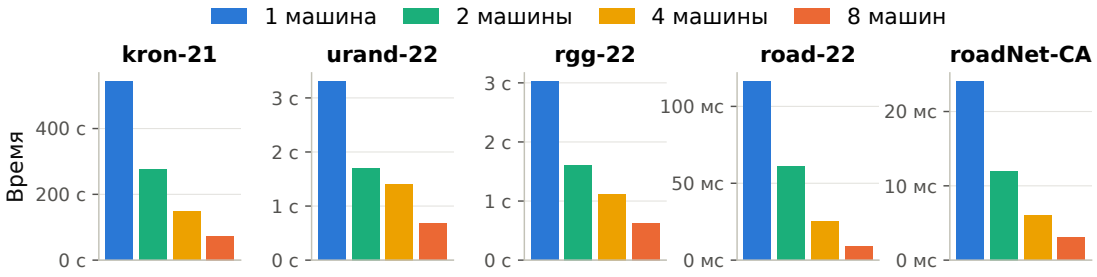
Код: `lonestar/analytics/distributed/bfs/bfs_push.cpp`; уровни — максимальное расстояние по scipy, раунды — `NumIterations`; у kron и urand на 1 раунд больше уровней — последний, пустой раунд.

PageRank: работа итерации делится поровну



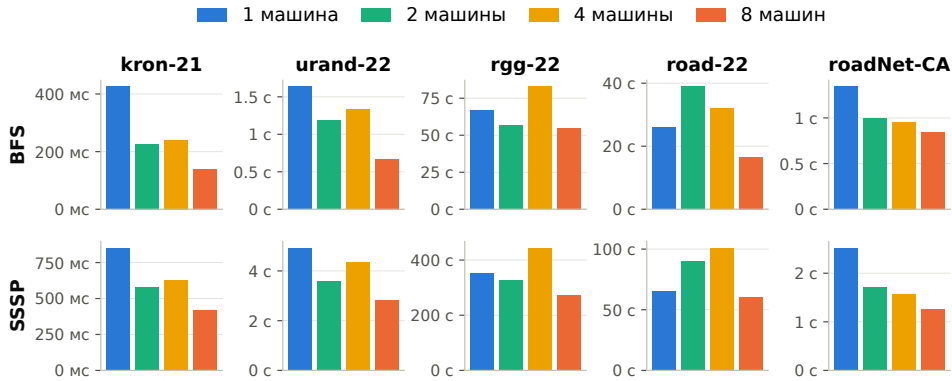
- Итерация проходит все рёбра, а разбиение делит рёбра поровну: каждая машина считает свою $1/p$.
- Gluon складывает остатки копий (`Reduce_add_residual`) — итерации те же, что на одной машине.
- У road-22 мало рёбер на копию: обмен сравним со счётом, выигрыш меньше.

TC: обмен только при построении графа



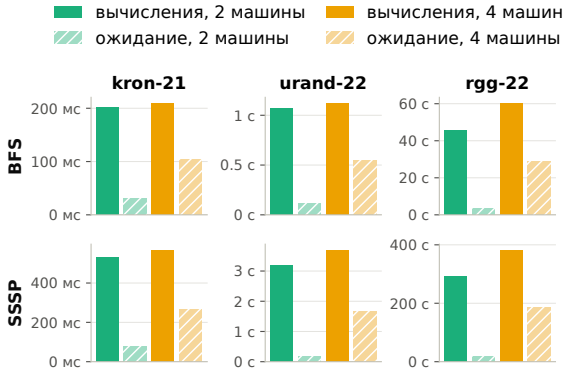
- Машина берёт свои вершины v и для каждого ребра $v \to w$ ищет соседей w в отсортированном списке v (`tc.cpp`). Рёбра ориентированы от меньшей степени к большей (`keepEdge`): треугольник считается один раз, а у хабов kron остаются короткие списки.
- Списки соседей чужих вершин машина получает при построении графа, до таймера.
- Во время счёта сообщений нет, в конце — одно `MPI_Allreduce` числа треугольников: работа делится между машинами без обмена и барьеров, поэтому время падает почти пропорционально числу машин.

BFS и SSSP: время почти не уменьшается



- Цена раунда — проход по всем вершинам, обмен и `MPI_Allreduce`; делится только проход.
- SSSP: вершина может получить неокончательное расстояние; опоздавшие на раунд обновления вызывают пересчёты, работа растёт.

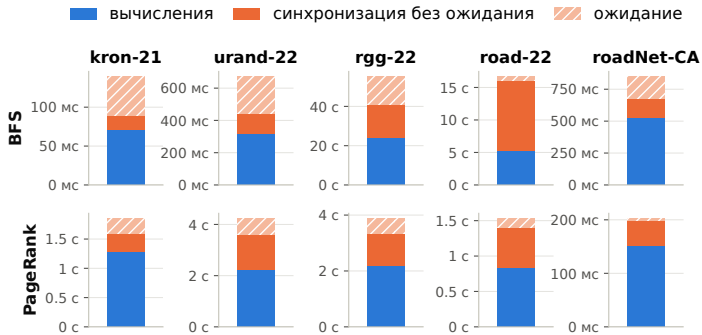
Почему 4 машины медленнее, чем 2



- Счёт самой медленной машины не уменьшается, хотя рёбер у неё вдвое меньше; остальные её ждут — ожидание растёт в разы.
- Копий вершин вдвое больше (при случайной нумерации вершина почти на каждой машине) — больше обмена.

Вычисления — таймер цикла (`BFS_0`, `SSSP_0`), ожидание — `Wait_*`; максимум по машинам.

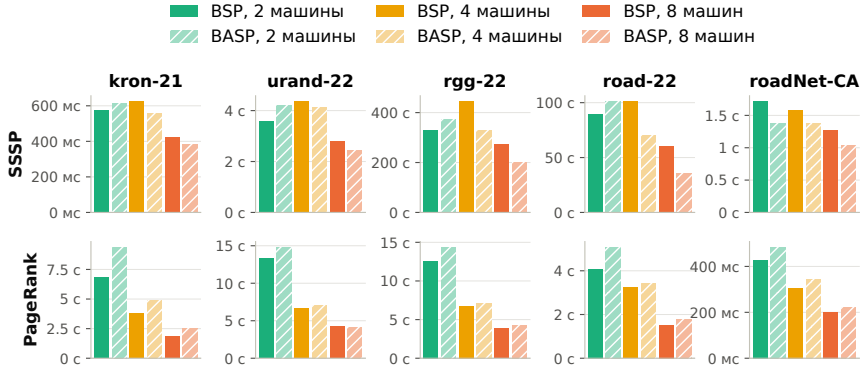
Куда уходит время на 8 машинах



- Ожидание (`Wait`) — машина ждёт тех, кто ещё считает: у BFS на kron, urand, rgg это заметная часть времени.
- У road-22 ожидания почти нет, а синхронизация — бóльшая часть времени: тысячи раундов, в каждом обмен. У PageRank ожидания мало: работа итерации поделена поровну.

Столбец — время алгоритма; синхронизация — таймеры Gluon `Sync_*`, ожидание — `Wait_*` (часть синхронизации), вычисления — остальное время; максимум по машинам.

Проверка: убираем барьеры — BSP против BASP



- BASP: нет `MPI_Allreduce` в раунде — быстрая машина не ждёт медленную, на 4–8 машинах дороги и rgg быстрее.
- На 2 машинах BASP почти везде медленнее: партнёр один, ждать почти некого, а без барьера локальных раундов и пересылок больше. Исключение — roadNet-CA: пересылок почти нет.
- PageRank медленнее: без барьера остатки приходят частями, итераций и сообщений больше.

Выводы

1. **Масштабируется то, где работа шага делится поровну и шагов мало**: TC и PageRank.
2. **Обходы упираются в раунды**: каждый — проход по всем вершинам, обмен со всеми машинами и общий барьер; без барьеров (BASP) обходы дорог быстрее.
3. **На 4 машинах медленнее, чем на 2**: счёт самой медленной машины не уменьшается, остальные её ждут.

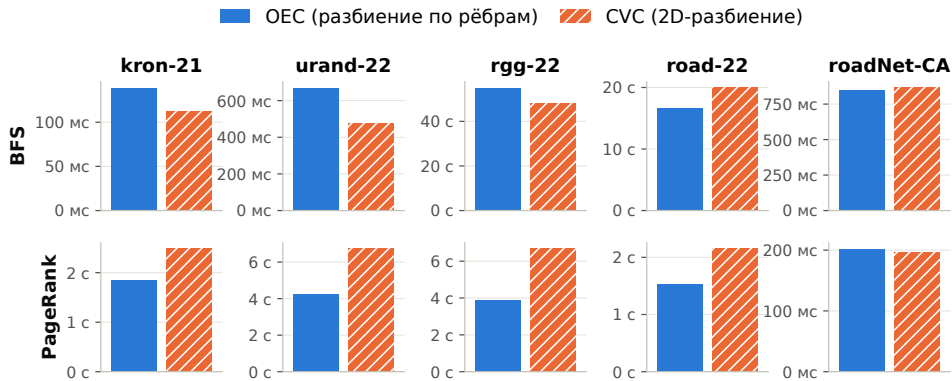
Код, данные и скрипт запуска на кластере: `github.com/…/graphs-2026`, каталог `block2-galois`.

Приложение: копии вершин и объём пересылок

Граф	Копий вершины			Пересылки за BFS	
	2 машины	4 машины	8 машин	2 машины	8 машин
kron-21	1.50	2.22	3.24	4 МБ	20 МБ
urand-22	2.00	3.95	7.05	38 МБ	172 МБ
rgg-22	2.00	3.92	6.85	1.9 ГБ	11.5 ГБ
road-22	1.81	2.54	3.01	1.7 ГБ	1.5 ГБ
roadNet-CA	1.01	1.01	1.02	2 МБ	5 МБ

- Копия есть на машине, если там хотя бы один сосед. При случайной нумерации машин с вершиной в среднем $1 + (p-1)(1-e^{-d/p})$ — совпадает с таблицей для urand и rgg до сотых.
- roadNet-CA пронумерован географически — копий почти нет.

Приложение: разбиение OEC против CVC (8 машин)



- CVC: решётка 2×4, копий не больше $2 + 4 - 1$ — пересылок меньше, BFS быстрее.
- PageRank: две фазы обмена (сведение и рассылка копиям) — медленнее.